

E09 — Non-overlapping Intervals (Greedy)

Experiment: 9 | LeetCode: #435 | CO: CO3, CO4 | BT Level: BT5

Problem Statement

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return the **minimum number of intervals you need to remove** to make the rest of the intervals non-overlapping.

Note that intervals which only touch at a point are non-overlapping. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Example 1:

Input: `intervals = [[1,2],[2,3],[3,4],[1,3]]`

Output: 1

Explanation: Remove `[1,3]` and the rest are non-overlapping.

Example 2:

Input: `intervals = [[1,2],[1,2],[1,2]]`

Output: 2

Explanation: Remove two `[1,2]` intervals.

Example 3:

Input: `intervals = [[1,2],[2,3]]`

Output: 0

Explanation: Already non-overlapping (touching at 2 is fine).

Concept: Greedy Interval Scheduling

This is equivalent to finding the **maximum number of non-overlapping intervals** (Activity Selection Problem), then:

```
min_removals = total_intervals - max_non_overlapping
```

Greedy Strategy: Sort intervals by **end time**. Always select the interval that ends earliest among those that don't overlap with the last selected.

Intuition: By picking the interval that ends earliest, we leave the maximum room for future intervals.

Greedy Proof Sketch

Claim: Sorting by end time and greedily selecting non-overlapping intervals is optimal.

Proof by exchange argument:

- Suppose an optimal solution selects interval X with a later end time, when interval Y (earlier end, same start or later) was available.
 - Swapping X for Y cannot reduce the number of selected intervals (Y ends earlier, leaving at least as much room).
 - Therefore, always picking the earliest-ending non-overlapping interval is at least as good as any other choice.
-

Implementation

```
def eraseOverlapIntervals(intervals):
    """
    Greedy: sort by end time, count overlapping intervals to remove.
    Time: O(n log n) | Space: O(1) (excluding sort)
    """
    if not intervals:
        return 0

    # Sort by end time
    intervals.sort(key=lambda x: x[1])

    max_keep = 1          # We can always keep at least 1 interval
    prev_end = intervals[0][1] # End time of last kept interval

    for i in range(1, len(intervals)):
        start, end = intervals[i]

        if start >= prev_end:
            # Non-overlapping: keep this interval
            max_keep += 1
            prev_end = end
        # else: overlapping - skip it (implicitly removing it)

    return len(intervals) - max_keep
```

Detailed Trace

Example 1: `[[1, 2], [2, 3], [3, 4], [1, 3]]`

Sort by end time:

`[[1, 2], [2, 3], [1, 3], [3, 4]]`

i	Interval	start	end	prev_end	Overlap?	max_keep
—	[1,2]	—	—	2 (initial)	—	1
1	[2,3]	2	3	2	$2 \geq 2 \rightarrow \text{keep}$	2, prev_end = 3
2	[1,3]	1	3	3	$1 < 3 \rightarrow \text{skip (overlaps)}$	2
3	[3,4]	3	4	3	$3 \geq 3 \rightarrow \text{keep}$	3, prev_end = 4

max_keep = 3, total = 4

Removals = $4 - 3 = 1$ ✓

Example 2: $[[1, 2], [1, 2], [1, 2]]$

Sort by end time:

$[[1, 2], [1, 2], [1, 2]]$ (all same)

i	Interval	start	prev_end	Overlap?	max_keep
—	$[1, 2]$	—	2	—	1
1	$[1, 2]$	1	2	$1 < 2 \rightarrow$ skip	1
2	$[1, 2]$	1	2	$1 < 2 \rightarrow$ skip	1

max_keep = 1, total = 3

Removals = $3 - 1 = 2$ ✓

Example 3: $[[1, 2], [2, 3]]$

Sort: $[[1, 2], [2, 3]]$

i	Interval	start	prev_end	Overlap?	max_keep
—	$[1, 2]$	—	2	—	1
1	$[2, 3]$	2	2	$2 \geq 2 \rightarrow$ keep	2

max_keep = 2, total = 2

Removals = $2 - 2 = 0$ ✓

Alternative: Sort by Start Time (Wrong!)

```
# Sorting by start time is NOT optimal:
#  $[[1, 10], [2, 3], [3, 4]]$  sorted by start =  $[[1, 10], [2, 3], [3, 4]]$ 
# Greedy picks  $[1, 10]$  first  $\rightarrow$  removes both  $[2, 3]$  and  $[3, 4] = 2$  removals
# But optimal: remove  $[1, 10]$ , keep  $[2, 3], [3, 4] = 1$  removal

# Correct answer: sort by END time
```

Test Suite

```
def test_erase_overlap():
    test_cases = [
        ([[1, 2], [2, 3], [3, 4], [1, 3]], 1),
        ([[1, 2], [1, 2], [1, 2]], 2),
        ([[1, 2], [2, 3]], 0),
        ([-100, -87], [-99, -44], [-98, -19], [-97, -26], [96, 99], [95, 99], [94,
```

```
for intervals, expected in test_cases:
    result = eraseOverlapIntervals([iv[:] for iv in intervals])
    status = "PASS" if result == expected else "FAIL"
    print(f"{status}: erase({intervals[:3]}...) = {result} (expected {expected})")

test_erase_overlap()
```

Complexity Analysis

Metric	Value
Time	$O(n \log n)$ — dominated by sorting
Space	$O(1)$ — only <code>prev_end</code> and count needed

Key Takeaways

- **Activity Selection Problem:** Classic greedy — sort by end time, greedily pick non-overlapping.
 - **Greedy choice:** The interval ending earliest leaves maximum room for subsequent intervals.
 - `min_removals = n - max_non_overlapping_intervals`
 - Intervals touching at a single point (`start == prev_end`) are **non-overlapping**.
 - Sorting by end time (not start time) is essential for the greedy to be correct.
-

References

- LeetCode #435 — Non-overlapping Intervals
- Cormen et al., *Introduction to Algorithms*, Ch. 16.1 (Activity Selection)
- Skiena, *The Algorithm Design Manual*, Ch. 8.1 (Greedy Algorithms)